ENSC 351 - Project Write-up

Dr. Matthew Stewart
December 3, 2025

Group Information:

- Ariful Shayun - ais4@sfu.ca
- Jeffrey Luan - jla881@sfu.ca
- Tyler Lee - tml9@sfu.ca
- Franco Obnamia - foa3@sfu.ca

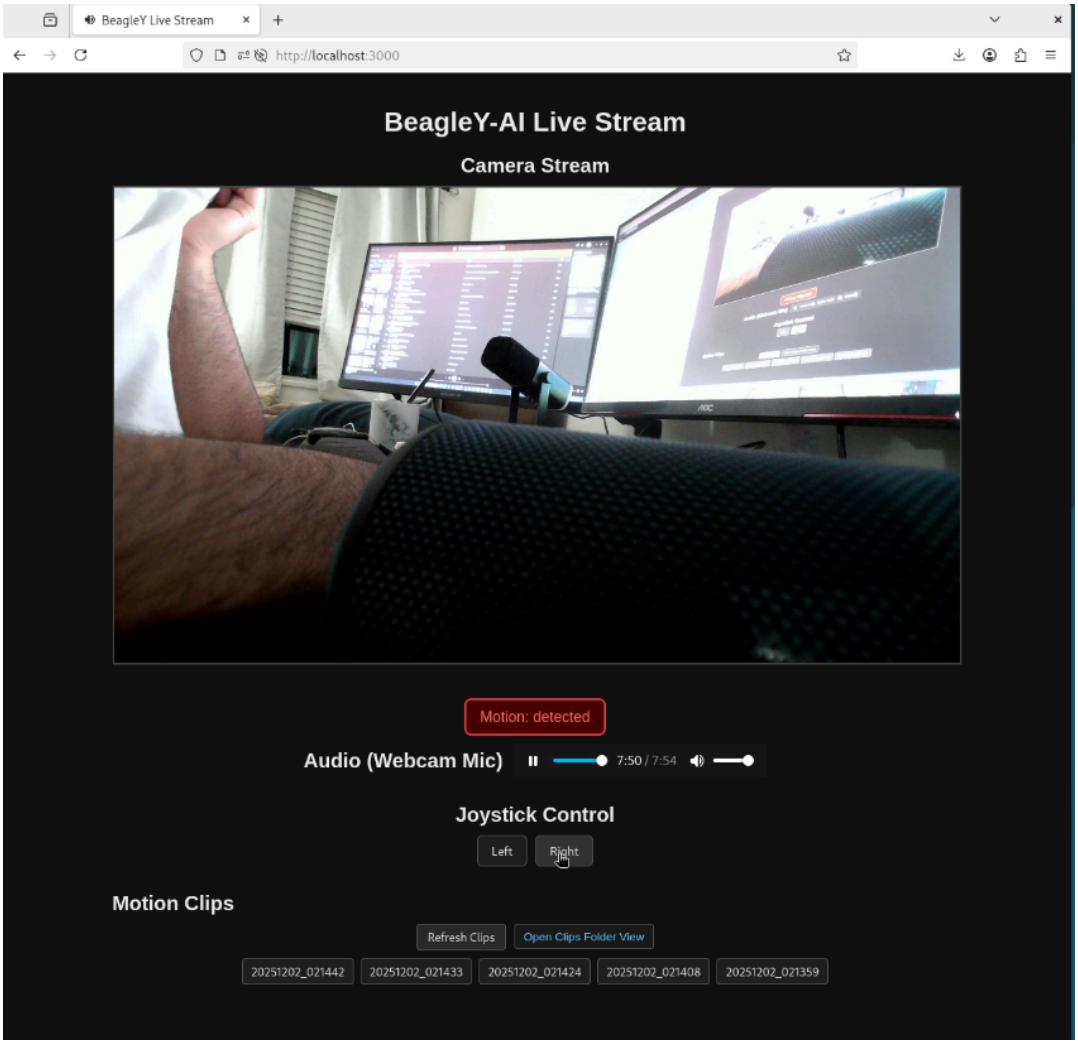
System Explanation: (1 page)

Our project, the Remote Video Camera and Display, integrates motion detection along with audio and video communication features, aiming to act as the conceptual idea behind the Ring Doorbell Camera. We will incorporate one-way video/audio from a camera to a website, with processing being done by the BeagleY-AI board using GStreamer and OpenCV. Additionally, we will implement many hardware features such as camera swivelling (joystick), doorbell (button and speaker) and lights (LED/Neopixel). Our system demonstrates enhanced safety and security through remote monitoring of visitors, suspicious activity, and vulnerable individuals.

Software:

Motion detection was done using OpenCV and is able to detect pixel-level changes as “motion” by decoding the output from the video. The pixel-level changes were calculated through grayscale extraction, which allowed us to detect whether a change in the frame was made. This was combined with the video/audio feed that was encoded/decoded using GStreamer and pushed to our host using UDP. Using WebSocket, we then pushed this data to our browser.

The website/browser is the primary feature of the project, which combines all features into a simplistic interface. First, the camera’s video feed is displayed on the website along with the audio feed, which can be played back using the slider and muted using the mute button. Both of these feeds have very minimal delays and are very clear in terms of quality. The website also contains controls for the servo motor, which have the same functionality as the joystick, sending UDP commands to the board which swivel the camera left/right. Motion detection is shown using a notification that displays whether motion is detected or idle, while also clipping that section and storing it in a local directory. This storage of video clips is automatically emptied as new clips are made, with the maximum capacity being 5 in our case, but during the testing phase, we were able to achieve up to 30 clips.



Some problems we ran into are the initialization stage of the microphone impacting sound quality for the first minute, before properly calibrating. We determined that this was a built-in software limitation of the webcam used. Some less significant/notable issues are different environments impacting motion detection and degradation in video/audio quality. This is due to using a laptop instead of a desktop, and to OpenCV not performing optimally. This was due to a lack of processing power, which we worked around by downscaling the resolution to a quarter of the original output.

Hardware:

In terms of hardware elements, we have multiple devices from our course assignments, such as a joystick and rotary encoder, as well as external ones, including an MG996R servo motor and a web camera. The joystick was also paired with the MCP3208 ADC, which we used in previous assignments.

The use of the web camera is, as expected, to provide video/audio input for our software to parse and analyze, as well as display on our website as a live feed. The camera we used provided adequate resolution and FPS for viewing on the browser.

Implementation of the joystick was to control the rotation of our web camera using the MG996R servo motor. The joystick input signal is converted using the ADC and read on the BeagleY-AI board via SPI. The servo motor then rotates in the direction the joystick is pushed, in our case, only left/right. Some limitations include a limited 180-degree angle of rotation and movement in the up/down directions, both of which are due to hardware/time constraints.

It features a doorbell, accurate to an actual doorbell from the Ring Camera. If a guest presses the doorbell, which in this case is the rotary encoder button, it outputs a doorbell chime.

Due to time constraints, we were not able to implement a UDP “doorbell” event to act as a notification to the website, indicating that a guest is at the door or a Neopixel/LED feature that flashes if someone presses the “doorbell.” We had spent significant time on the Neopixel, but were not able to implement it into our project.

Feature Table: (1-2 pages)

Description	Host/Target	Completeness	Code	Author(s)	Notes
Website Backend	H	5	JS, Node.js	Ariful Shayun	
Website Frontend	H	5	HTML, CSS, JS	Ariful Shayun	
Camera (video/audio feed)	T	5	GStreamer	Ariful Shayun, Tyler lee	Camera feed relied on motion detection resolution
Live video/audio streaming	H	4.5	GStreamer	Ariful Shayun	Calibration issues, poor laptop performance.
Motion Detection	H	4	C++, OpenCV	Ariful Shayun, Tyler Lee	Affected by lighting conditions and environments
Video Clipping	H	4		Ariful Shayun	
Servo Motor	T	5	C	Jeffrey Luan	
Joystick	T	5	C	Jeffrey Luan	Based on as3 code
Rotary Encoder Button	T	5	C, ALSA	Franco Obnamia	Doorbell, based on as3 code
Speaker	T	5	C, ALSA	Franco Obnamia, Jeffrey Luan	Doorbell chime
NeoPixel	T	2	C	Franco Obnamia, Tyler Lee	Could not implement R5 for Neopixel.
LED	T	2.5	C	Franco Obnamia	Had code running separate from main project code.

As seen in the table above, tasks were divided equally among our group based on each member's strengths and divided equally among our group members, with many contributions from each member for each part, even if not listed above.

Extra Hardware & Software Used: (0.5 pages)

The extra hardware we used outside of what we were given from class was an external camera and a servo motor acting as the swivel of the camera. The camera was used to integrate motion detection and as an active surveillance for a doorbell camera. The servo motor allowed the user to move the camera left and right to see a wider field of view. Additionally, we used an external speaker to output the doorbell chime sound for better audibility.

The extra software used included GStreamer and OpenCV. GStreamer was crucial for processing the live camera feed. This framework encoded raw video/audio from the camera and then decoded it on the BeagleY-A board for further processing. OpenCV was integrated for all motion detection with input from the GStreamer feed.